

第 1 讲

Introduction

目录

1 课程框架与技术背景	1
1.1 课程的技术定位	1
1.2 课程形成背景与计算平台脉络	2
1.3 课程规则与技术导论的逻辑关系	2
2 课程运行框架、作业流程与学术规范	3
2.1 授课形式与支持体系	3
2.2 教材与技术文档	4
2.3 典型周节奏与提交时间	4
2.4 评分结构与考核方式	5
2.5 实验的端到端 workflow	6
2.6 Delta 超级计算机与作业调度	6
2.7 Delta 账户与登录步骤	8
2.8 课程人工智能问答系统的技术机制	9
2.9 允许与禁止的协作和人工智能使用	9
3 从摩尔定律时代到后登纳德时代	10
3.1 两条经常被混淆的缩放规律	11
3.2 “自动性能增长”对软件的影响	11
3.3 专用计算机在早期为何难以长期保持优势	12
3.4 轻松性能增长的结束	12
3.5 后登纳德时代的三条主要性能路径	13
3.6 并行性与异构性的共同转向	14
4 中央处理器与图形处理器的设计取向	15
4.1 先区分延迟与吞吐量	15
4.2 中央处理器：面向低延迟的设计	16
4.3 图形处理器：面向高吞吐量的设计	17
4.4 芯片面积分配反映了不同目标	18
4.5 “核心”一词在两类处理器中并不等价	19
4.6 高性能系统通常同时使用中央处理器与图形处理器	19
5 图形处理器的起源、统一计算设备架构与应用扩张	20
5.1 图形工作负载天然具有大规模数据并行性	20
5.2 2007 年的统一计算设备架构转折	21
5.3 为什么沿用图形处理器，而不是从零设计另一种吞吐处理器	22
5.4 数据中心与人工智能推动通用图形处理器计算扩张	22
5.5 人工智能计算需求的数量级增长	23
5.6 超级计算机中的加速器普及	24
5.7 并行编程框架之间的关系	25
6 并行化工作流与核心性能挑战	26
6.1 从“程序太慢”到可验证的并行实现	26
6.2 挑战一：大规模并行要求规则性	27
6.3 挑战二：负载均衡	28

6.4	挑战三：全局内存带宽的理想值与实际值	29
6.5	挑战四：冲突访问导致串行化与延迟	29
6.6	四类挑战之间的耦合	30
7	阿姆达尔定律：整体加速比的串行上界	30
7.1	问题设定与符号	30
7.2	总执行时间的推导	31
7.3	并行部分无限快时的上界	31
7.4	示例：90% 可并行，局部加速 1000 倍	32
7.5	串行比例对上界的高度敏感性	33
7.6	为什么先做性能剖析	33
7.7	模型边界与现实开销	34
8	综合框架与关键结论	34
8.1	硬件演化到软件责任的完整链条	34
8.2	选择中央处理器或图形处理器时的判断顺序	35
8.3	高频概念辨析	35
8.4	“高性能”的操作性定义	36
8.5	阅读与操作任务	36

1 课程框架与技术背景

内容概览

课程运行框架包括课程目标、资源平台、作业流程、评分方式、计算环境以及学术诚信要求。单核频率不再持续快速增长之后，现代计算系统转向**并行性 (parallelism)**、**异构性 (heterogeneity)** 与 **处理器专用化 (processor specialization)**，图形处理器也成为这一转向中的核心加速器。

相关硬件趋势与软件责任构成以下因果链：

晶体管密度持续增加 + 频率缩放趋于停滞
⇒ 更多晶体管无法自动转化为更快的单线程执行
⇒ 多核、向量化与专用加速器成为主要性能路径
⇒ 软件必须显式暴露并行性，并控制负载、访存与串行瓶颈。

1.1 课程的技术定位

课程名称为应用并行编程，核心对象是**大规模并行处理器 (massively parallel processor)**，主要以支持统一计算设备架构的英伟达图形处理器为实验平台。课程并不把处理器结构本身作为最终目标；架构知识只学习到足以解释程序性能的深度。换言之，处理器结构、编程接口、并行算法模式和性能测量在本课程中始终服务于同一个目标：把一个具有足够并行性的计算映射到实际硬件上，并使映射后的程序获得可解释、可复现且可扩展的性能。

课程目标包含以下五类技术内容：

- 并行编程的基本模型与术语；
- 常见并行算法的原则与模式；
- 编程接口、工具链与实现技巧；
- 影响性能的处理器结构特征与约束；
- 以人工智能负载为代表的高价值应用。

关键结论

并行程序的质量不能只用“能否运行”衡量。一个合格的并行实现至少同时满足四项要求：结果正确、性能足够高、代码可维护、算法与实现能适应后续硬件代际。仅通过增加线程数得到一次性的速度提升，并不等于获得了可扩展的并行程序。

1.2 课程形成背景与计算平台脉络

美国国家超级计算应用中心的图形处理器系统建设构成了本课程的计算平台背景。课程教师长期从事高性能计算、计算加速器以及机器学习系统研究。三代具有代表性的系统分别是：2008年建设的加速集群使用较早期的图形处理器；2018年建设的面向人工智能的集群使用 V100 图形处理器；2024年建设的 DeltaAI 系统采用基于 GH200 超级芯片的 Grace-Hopper 节点。课程实验实际使用较早的 Delta 超级计算机，而不是 DeltaAI。

图形处理器属于更大的系统演化趋势：高性能计算系统逐步从以通用中央处理器为绝对中心的结构，转向由中央处理器负责控制与串行工作、由一个或多个加速器承担大规模并行计算的异构结构。

1.3 课程规则与技术导论的逻辑关系

课程运行规则与技术导论看似是两组独立内容，实际存在直接联系：

- 课程要求在 Delta 上编译并通过作业调度系统运行，是因为真实超级计算机由大量用户共享，计算资源必须被隔离和调度。
- 实验强调自己编写代码，是因为期中考试要求在纸面上完成与实验相近的程序片段；依赖自动代码生成无法形成独立实现能力。
- 项目同时评价功能与性能，是因为并行程序的价值来自正确性与性能的共同满足。
- 硬件代际快速变化，因此并行程序需要具备可移植性、可扩展性和可维护性，不能只适用于某一张图形处理器。

评估并行化方法时需要同时回答两个问题：第一，该方法为什么能提高吞吐量；第二，该方法在真实共享系统、未来硬件和完整应用中是否仍然成立。

2 课程运行框架、作业流程与学术规范

2.1 授课形式与支持体系

课程以线下授课为主，课堂同时录制并上传至学校媒体平台。实验又称**机器问题 (Machine Problem, MP)**，项目由学生在课外独立或按指定团队形式完成；两次期中考试均在线下进行。助教团队分别承担实验、项目和答疑工作，办公时间从第二周开始，其时间表由课程管理平台公布。

课程支持体系由多个平台共同构成。每个平台只承担一部分功能，因此应按任务类型选择入口。

平台	主要用途	使用要点
课程管理平台 (Canvas)	教学大纲、课件、录像链接、课程文件、项目报告与成绩	课件通常在上课前上传；最终成绩在此汇总。 打开课程页面 。
学校媒体平台 (Illinois Media Space)	课堂录像	录像发布到课程专用频道。 打开录像频道 。
代码托管平台 (GitHub)	实验与项目的基础代码、开发仓库和最终代码提交	先同步发布代码，再在 Delta 上编辑、编译和运行，最后把待评分版本提交到仓库。 打开发布仓库 。
在线评测平台 (PrairieLearn)	实验配套测验与概念问题	应在代码部分完成之后作答，因为部分问题依赖实际运行结果。
课程讨论平台 (Campuswire)	公告、学生问答、教师与同学共同讨论	面向班级显示时可采用匿名形式；不能把它当作提交受保护代码的渠道。 打开讨论区 。
课程人工智能问答系统 (course AI chatbot)	课程规则、统一计算设备架构概念、接口、编译与运行错误等问题	推荐先询问该系统；若结果不充分，再使用讨论区或办公时间。链接以课程管理平台的当期入口为准。

2.2 教材与技术文档

首选教材为 Wen-mei Hwu、David Kirk 与 Izzat El Hajj 编写的《大规模并行处理器编程：实践方法》(*Programming Massively Parallel Processors: A Hands-on Approach*) 第五版，2026 年出版。第四版于 2022 年出版，也被课程接受，并可通过学校订阅在线阅读。第三版及更早版本由于内容老化，不建议使用。

统一计算设备架构的主要文档包括：

- 《统一计算设备架构 C 编程指南》(*CUDA C Programming Guide*);
- 《统一计算设备架构 C++ 最佳实践指南》(*CUDA C++ Best Practices Guide*)。

Delta 环境安装 CUDA 13.2，文档参考涉及 13.2 与 13.3 两个版本。阅读文档时需核对文档版本与安装版本；实际使用时应以课程管理平台的当期公告为准。

2.3 典型周节奏与提交时间

前三周的示例安排如下：

表 2 前三周典型节奏

周次	课堂内容	课外任务与截止项
第 1 周	周二：第 1 讲“导论”；周四：第 2 讲“统一计算设备架构导论”	配置代码仓库与计算机访问权限。
第 2 周	周二：数据并行模型；周四：统一计算设备架构内存模型	实验 0：设备查询示例。
第 3 周	周二：继续讲解内存模型；周四：性能考虑	实验 1：向量加法示例。

部分周次会用项目检查点替代实验提交；项目被拆分为三个检查点，并采用逐步完成的交付方式。

所有实验和项目里程碑的正式截止时间均为**美国中部时间周五晚 8:00**。课程同时提供三天默认宽限期：在周一晚 8:00 之前提交仍可获得满分，且不需要提前通知。宽限期的制度含义必须准确理解：

- 正式截止时间仍是周五晚 8:00，周一不是新的常规截止时间；
- 如需超过三天的额外延长，必须在周五晚 8:00 之前给负责该作业的助教发邮件；

- 周五晚 8:00 之后提出的额外延长请求不予考虑，是否批准由课程团队根据特殊情况决定；
- 周一晚 8:00 时仓库中的代码被视为最终版本。即使较早提交过正确版本，评分也会取该时刻检出的仓库状态。

需要特别注意

共享超级计算机可能因系统维护或队列拥堵而延迟作业启动。宽限期为这类不可控情况提供缓冲，但它不能替代提前开始。若大量学生在周五集中提交，等待计算节点本身就可能消耗数小时。

2.4 评分结构与考核方式

课程总评由三部分组成：

表 3 总评构成

组成	权重	具体规则
期中考试	50%	两次期中考试各占 25%；第一次约覆盖前 11 讲，第二次约覆盖其余讲次。课程没有期末考试；期中考试为三小时纸笔考试，并要求手写与实验相近的代码。
实验	25%	自动数据集通过情况约占实验成绩的 90%，同时考查配套问题的正确性；最低的一个计分实验成绩被舍弃；实验 0 和实验 1 不计入总评。实验数量与各项权重以课程团队发布的详细量规为准。
项目	25%	演示、功能与编码风格约占项目分数的 50%；在完整功能成立的前提下，性能约占另外 50%。详细量规将在之后公布。

项目形式包括两种：一种围绕卷积神经网络若干层的实现；另一种由两至三人组队，实现大语言模型或变换器中的若干层。无论选择哪类项目，功能完整性都先于性能评分。一个输出错误但运行很快的程序不构成高性能实现，因为其计算语义已经改变。

纸笔考试与实验之间形成直接约束。实验中出现的程序结构可能在考试中以手写代码形式再次出现，因此完成实验的目标包含两层：一是获得正确输出，二是建立不依赖代码生成器或他人实现的独立编程能力。

2.5 实验的端到端 workflow

实验采用下列端到端流程：

1. 从课程发布仓库取得基础代码，并按说明把发布内容合并到个人仓库；
2. 在 Delta 的登录节点上编辑与编译代码；
3. 通过作业调度系统把程序提交到计算节点执行，不能在登录节点上直接运行重计算任务；
4. 根据正确性测试与运行结果继续调试，直到代码满足作业要求；
5. 将最终版本提交到代码托管仓库，供评分流程检出；
6. **代码部分完成后**，再回答在线评测平台中的配套问题。

最后一步的顺序不是形式要求。实验问题往往询问实际输出、性能现象或实现细节；在程序尚未完成时作答，会把推测当成测量结果，也更容易忽略题目中隐含的前提。

2.6 Delta 超级计算机与作业调度

Delta 是由美国国家超级计算应用中心运行、通过高等科学与工程研究计算环境分配资源的超级计算机。课程使用其中配备 A40 图形处理器的子系统。四卡计算节点规格如下。

NCSA Delta Supercomputer



- Delta is a dedicated, [ACCESS](#)-allocated resource designed by HPE and NCSA, delivering a highly capable GPU-focused compute environment for GPU and CPU workloads.



4-Way A40 GPU Compute Node Specs	
Specification	Value
Number of nodes	100
GPU	NVIDIA A40
GPUs per node	4
GPU Memory (GB)	48 DDR6
CPU	AMD Milan
CPU sockets per node	1
Cores per socket	64
Cores per node	64
Clock rate (GHz)	~ 2.45
RAM (GB)	256

ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1 Delta 四卡 A40 子系统

表 4 四卡 A40 计算节点规格

项目	数值
节点数量	100
图形处理器型号	英伟达 A40
每节点图形处理器数量	4
每卡显存	48 GB
中央处理器	AMD Milan
每节点中央处理器插槽数	1
每插槽核心数	64
每节点核心总数	64
近似时钟频率	2.45 GHz
主存容量	256 GB

Delta 是多用户共享系统。用户首先连接到**登录节点 (login node)** 或**头节点 (head node)**，在这里管理文件、编辑代码和编译程序。实际计算通过**作业调度系统 (job scheduler)** Slurm 提交到**计算节点 (compute node)**。调度器根据请求的资源、队列状态与系统策略决定作业何时启动。

该共享系统具有以下运行特性：

- 用户可能登录到多个登录节点中的任意一个；这些节点共享文件系统与调度队列，因此登录节点编号变化不影响文件可见性。

- 系统繁忙时，作业可能等待数分钟、数小时，项目级大任务甚至可能等待更久。
- 课程作业通常请求一张图形处理器，而一个计算节点包含四张 A40，因此同一节点上最多可同时存在若干用户作业。
- 调度器对分配给不同作业的图形处理器进行隔离；一个作业不能使用分配给另一个作业的设备。
- 登录节点面向许多并发用户，不能把它当作绕过调度器的计算节点。

关键结论

在共享超级计算环境中，“程序运行时间”与“从提交到拿到结果的墙钟时间”不是同一个量。前者是作业实际执行时间，后者还包含排队等待。课程截止时间约束的是完整 workflow，因此排队风险必须通过提前提交来管理。

2.7 Delta 账户与登录步骤

账户建立需要经过高等科学与工程研究计算环境和 Delta 两层系统，不能在截止日前临时完成。具体步骤如下：

1. 在[账户注册页面](#)创建用户；已有账户者继续使用原用户名，不应重复注册。
2. 新建账户时把“伊利诺伊大学厄巴纳-香槟分校”选作[身份提供方 \(Identity Provider\)](#)，以学校凭据完成认证。
3. 记录由系统分配的用户标识，并通过[课程表单](#)提交给课程团队。
4. 等待课程团队把用户加入课程资源组，再等待 Delta 端完成账户创建。典型等待时间为 24–48 小时，完整传播过程可能更长。
5. 收到确认邮件后，通过安全外壳协议连接：

```
ssh yourusername@login.delta.ncsa.illinois.edu
```

6. 登录需要 Duo 双因素认证；随后克隆课程仓库并按仓库说明配置环境。

2.8 课程人工智能问答系统的技术机制

课程问答系统采用**检索增强生成 (Retrieval-Augmented Generation, RAG)**。它不只依赖语言模型内部参数回答问题，而是先从课程材料中检索相关片段，再把这些片段作为上下文交给语言模型。

建模说明

其基本数据流可表示为

课程文档 → 分段 → 嵌入向量 → 向量数据库，
 学生问题 → 问题嵌入 → 相似片段检索，
 → 上下文拼接 → 大语言模型 → 回答与来源指向。

其中，嵌入向量 (embedding) 用数值向量表示文本语义，向量数据库 (vector database) 用于相似性检索，大语言模型 (large language model) 根据检索上下文生成回答。向提示词中放入大量课程上下文称为**上下文填充 (context stuffing)** 或**超长提示构造 (mega-prompting)**。系统可通过应用程序编程接口调用模型，也可使用本地运行的开源模型。

检索增强只能提高回答与课程材料的一致性，不能保证每个答案都正确。系统可能遗漏题目隐含假设、检索到不完整片段，或在生成过程中引入错误。课程明确规定：学生对最终代码与答案承担责任，不能把“问答系统给出了该答案”作为重新评分的依据。

2.9 允许与禁止的协作和人工智能使用

课程允许概念层面的讨论，也鼓励学生利用问答系统学习接口与技术；同时严格禁止把实现责任转移给他人或人工智能工具。边界可按下表理解。

允许	禁止
与同学讨论思路、算法方向、所用函数以及调试策略；接受修过课程者的口头建议。	查阅往年作业实现或网络上发布的本课程作业答案。
询问统一计算设备架构接口的含义，例如内存复制函数的作用、内核函数的调用方式。	要求人工智能系统直接生成某个实验或项目的实现代码。
询问编程技术，例如共享内存数组如何声明、运行时如何指定动态共享内存大小。	复制超过一行左右的非平凡代码；让他人提供代码同样会受到处罚。

允许	禁止
询问并行编程概念与性能优化原理，例如分块为什么能改善全局内存带宽。	阅读他人代码或人工智能生成的代码后，再离开原文“独立重写”；课程把这种行为也视为复制。
询问编译错误、运行时错误的含义，并学习系统化调试方法。	把完整程序粘贴给问答系统，让系统代替自己定位并修复错误。
进行期中复习与一般课程规则查询。	把实验测验题直接粘贴给问答系统，或在考试中给予、接受帮助。
逐行理解并按当前问题改写课堂明确提供的代码示例。	故意规避实验要求，或只做表面变量改名、语句重排来掩盖复制。

“非平凡代码”通常指超过一行左右的实现。代码相似性检测不只比较变量名，也能识别重排、改名等表面变化。允许引用课堂示例的前提是逐行理解并针对当前程序进行实质性改写，而不是整段搬运。

学术不端的后果为：第一次在相应作业或考试上记零分，并通过学校的 FAIR 系统向学院报告；重复违规将导致课程自动不及格。提供代码者与接收代码者都可能受到相同处罚，因为相似性检测并不能可靠判断谁最先写出该实现。

需要特别注意

课程对人工智能工具的限制不是“完全禁止使用”。允许用途集中在概念、接口、错误解释与复习；禁止用途集中在替代学生完成受评分实现。判定边界时应问：该工具是在帮助理解问题，还是在产生本应由学生独立完成的答案或代码？

3 从摩尔定律时代到后登纳德时代

3.1 两条经常被混淆的缩放规律

现代并行计算的硬件动因必须从两条历史趋势区分开来。

表 6 摩尔定律与登纳德缩放的作用区分

概念	核心含义	对性能的历史影响
摩尔定律 (Moore's Law)	制造特征尺寸不断缩小，使单位面积可容纳的晶体管数量约每 18–24 个月翻倍。它是经验性“定律”，也可视为产业目标。	提供更高的元件密度，使芯片能够容纳更多功能单元、缓存和控制逻辑。它本身并不等价于单线程性能翻倍。
登纳德缩放 (Dennard Scaling)	随器件尺寸缩小，晶体管能够在更高时钟频率下工作，是推动频率上升的物理基础。	使单个处理器核心能够更快执行同一指令流，是早期“相同代码自动变快”的重要来源。

大众叙述常把处理器性能增长统称为“摩尔定律”，但历史上的快速性能增长实际来自摩尔定律与登纳德缩放的共同作用。晶体管数量增加提供了更复杂的处理器结构，时钟频率提高则直接缩短许多指令的执行时间。两者叠加后，通用计算机在约三十年内表现出大致每 18 个月翻倍的性能趋势。

3.2 “自动性能增长”对软件的影响

在这一时期，软件常能在不改变算法与并行结构的情况下获得显著加速。一个今天运行过慢的串行程序，经过一到两个处理器代际后，只需重新编译或迁移到新机器，就可能明显变快。由此形成了三项长期软件预期。

3.2.1 同一份软件随硬件代际自动加速

程序不需要显式使用更多线程，也能受益于更高频率、更好的流水线与更强的单核结构。在这种历史环境中，“代码不够快，等待两年”一度可以成立；它不是现代系统中的可靠开发策略。

某些程序较早暴露了这一策略的边界。若程序主要受内存访问延迟 (*memory-access latency*) 限制，例如部分稀疏、非结构化或图算法，处理器算术速度提高并不会按相同比例缩短运行时间。此类程序需要的数据访问不规则，缓存难以持续命中，因此内存系统可能先成为瓶颈。

3.2.2 内存从稀缺资源变为多数应用中的充足资源

芯片密度增加也扩大了可用内存容量。早期程序常需要显式把数据分块，在磁盘与主存之间来回搬运；随着系统能够提供数百吉字节乃至更大容量，许多应用可以把完整数据集常驻内存。这一变化直接改变了算法设计：过去为节省内存而使用的复杂外存策略，在新的容量条件下可能不再必要。

3.2.3 软件可移植性成为默认期待

如果相同程序可以在多个处理器代际上继续运行并自动加速，那么软件稳定性和可移植性(*portability*)自然具有很高价值。应用代码可以长期演进，而不必每次为了新硬件彻底重写。可扩展且可移植的软件能够跨越多个硬件代际；现代并行软件必须通过更主动的设计才能维持这种寿命。

3.3 专用计算机在早期为何难以长期保持优势

专用处理器可以针对某类计算取得初始性能优势，但在通用处理器性能约每 18 个月快速增长的时期，该优势可能在数年内被下一代通用微处理器追平。专用芯片的设计、制造与软件生态成本很高，如果其目标市场不足以支撑持续迭代，就难以与通用处理器的规模经济竞争。

这种现象称为“杀手级微处理器的冲击”(Attack of the Killer Micros)：许多试图领先于摩尔定律与登纳德缩放组合速度的计算机公司，最终被快速演化的通用微处理器压缩了生存空间。图形处理器之所以比许多其他专用加速器更容易形成长期生态，也与这一背景有关。

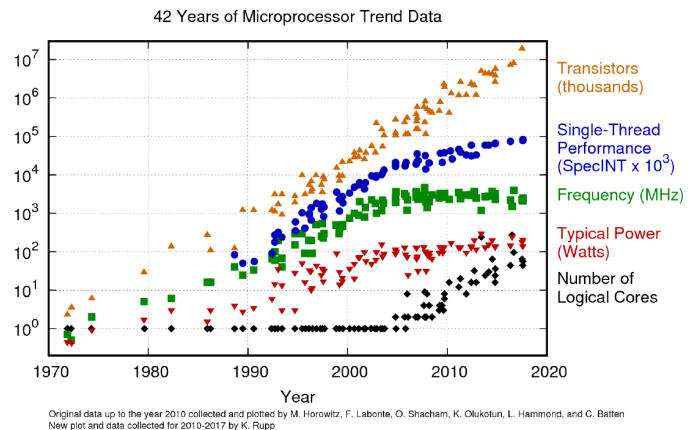
3.4 轻松性能增长的结束

约在 2005–2006 年，登纳德频率缩放停止延续原有趋势。过去可以忽略的漏电流(*leakage current*)变得重要，功耗与散热约束使时钟频率无法继续按历史速度增加。处理器频率大体停留在约 2.5 GHz to 3.5 GHz 的范围；近二十年后，频率数量级仍没有发生根本变化。

与此同时，晶体管数量仍在增加，尽管摩尔定律本身也在放缓。于是出现了新的设计矛盾：芯片可以容纳更多晶体管，却不能简单地把它们全部用于提高一个核心的时钟频率。图 2 中，晶体管数量继续上升，而频率曲线在 2000 年代中期后明显趋平；逻辑核心数量随后开始增长。

End of Easy Performance Gains

- Dennard (Frequency) Scaling ended around 2005-2006
 - Previously neglected effect, leakage current, became important
 - Clock speeds stagnated at 2.5-3.5 GHz
 - Almost 20 years later, clock speeds are nearly the same
- But the expectation of doubling of performance was strong
 - And Moore's "Law", though slowing down, is still providing higher component density
- Maintaining performance expectation required doing something different
 - Instruction parallelism (vectors)
 - Multiple cores per "processor"
 - Specialization, especially to vectors/streams/GPUs



Slides courtesy of Prof. William Gropp, NCSA Director



NCSA | NATIONAL CENTER FOR SUPERCOMPUTING APPLICATIONS

23

图2 单纯依靠频率获得性能增长的时代结束

关键结论

登纳德缩放结束后的关键变化是：硬件仍可能提供更多计算资源，但串行程序不会自动使用这些资源。新增晶体管被组织为更多核心、更多向量执行单元或专用加速器之后，软件必须暴露与之匹配的并行工作，才能把硬件规模转化为应用性能。

3.5 后登纳德时代的三条主要性能路径

维持性能增长主要有三种方向。

3.5.1 指令级与向量并行性

向量执行 (vector execution) 让一条指令对多个数据元素执行相同运算。其基本思想是把规则、重复的数据运算打包，使同一控制流驱动多个算术通道。后续图形处理器设计继承并扩大了这种“相同操作作用于不同数据”的思想。

3.5.2 多核处理器

当单个核心无法继续大幅提高频率时，可以在同一处理器中放置多个核心。每个核心执行自己的指令流，共享部分缓存、内存与互连资源。多核设计提高了芯片可并行执行的工作量，但单线程程序通常仍只使用一个核心，因此程序需要被改写为多线程或多进程形式。

现代处理器可包含上百个核心。即使硬件核心数从 1 增长到 100，一个只含单一串行指令流的程序也不会自动获得 100 倍加速。核心数量是可用资源，不是无条件性能保证。

3.5.3 处理器专用化

通用中央处理器追求“几乎所有代码都能运行，并在多数工作负载上获得尚可到优秀的表现”。专用处理器则主动放弃一部分通用性，把更多芯片面积用于某类操作或数据流。处理器专用化包括以下方向：

- 使用更简单的轻量级核心并增加核心数量，例如高性能计算中的蓝色基因系统与至强融核；
- 针对向量、流式数据或图形运算构建专用结构；
- 在移动芯片中同时配置中央处理器、图形处理器和神经网络处理单元 (Neural Processing Unit, NPU)，分别承担通用控制、图形与特定机器学习运算；
- 在超级计算机中由通用中央处理器协调，由图形处理器承担大规模数值计算。

3.6 并行性与异构性的共同转向

异构计算 (*heterogeneous computing*) 是指同一系统包含多种结构不同、性能特征不同的处理器，并把应用的不同部分映射到最合适的设备。它与并行性相互依赖：专用加速器往往通过大量并行执行单元取得吞吐量优势，而应用必须提供足够规则的并行任务才能利用这些单元。

移动芯片 Apple A14 Bionic 与橡树岭国家实验室 Frontier 百亿亿次级系统都采用加速器架构，说明这种结构从最小规模的移动设备一直延伸到最大规模的超级计算机。

The Trend is Across the Board



- From the smallest to the largest, computing today involves accelerators architectures



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图3 加速器架构贯穿移动设备与超级计算机

这一转向改变了软件责任分配。在早期“自动性能增长”环境中，硬件升级可以替软件完成大量加速工作；在后登纳德时代，硬件只提供并行资源与专用能力，程序员必须识别可并行部分、选择可扩展算法、组织数据并消除瓶颈。

4 中央处理器与图形处理器的设计取向

4.1 先区分延迟与吞吐量

比较中央处理器和图形处理器之前，必须先区分两个性能指标。

- 延迟 (latency)** 描述单个任务从开始到完成所经历的时间。若任务在时刻 t_{start} 开始、在 t_{finish} 完成，则

$$L = t_{\text{finish}} - t_{\text{start}}$$

延迟越低，单个请求越快得到结果。

- 吞吐量 (throughput)** 描述单位时间内完成的任务数量。若时间区间 Δt 内完成 N 个任务，则平均吞吐量为

$$R = \frac{N}{\Delta t}$$

吞吐量越高，系统在一段时间内处理的总工作越多。

低延迟与高吞吐量并不等价。一个流水线可以让单个任务经过多个阶段，因此单任务延迟较长；当流水线填满后，各阶段同时处理不同任务，系统仍可在每个周期完成一个或多个结果，从而获得高吞吐量。

建模说明

设一条理想流水线包含 d 个阶段，时钟频率为 f 。单个任务的流水线延迟约为 d/f ，但流水线达到稳态后，每条执行通道的理想结果产生速率可以接近 f 。增加流水线深度可能不降低单任务延迟，却能支持更高的稳态工作速率，从而形成图形处理器“长延迟、高吞吐量”的组合。

4.2 中央处理器：面向低延迟的设计

中央处理器采用面向延迟的设计 (*latency-oriented design*)，主要优化少量指令流的快速完成，其芯片面积通常分配给以下结构。

4.2.1 少量但功能强的算术逻辑单元

算术逻辑单元 (*Arithmetic Logic Unit, ALU*) 执行整数、逻辑及部分数值运算。中央处理器核心包含较少但较复杂的执行单元，目标是降低单条指令或短指令序列的完成时间。复杂的乱序执行与多种执行端口也属于同一低延迟取向。

4.2.2 较大的缓存层次

主存访问延迟远高于算术操作。中央处理器投入大量芯片面积构建多级**缓存** (*cache*)，把频繁访问的数据保留在靠近执行单元的位置。缓存命中时，原本较长的主存访问被转换为较短的缓存访问，从而降低单线程停顿。

4.2.3 复杂控制逻辑

中央处理器使用复杂控制结构减少依赖与分支带来的等待：

- **分支预测** (*branch prediction*) 提前猜测控制流方向，尽量避免处理流水线因等待分支结果而空闲；
- **数据转发** (*data forwarding*) 把某条指令刚产生的结果直接送给后续相关指令，减少等待数据写回再读取的时间；

- 适度多线程用于隐藏相对较短的延迟，但线程数量通常远少于图形处理器。

中央处理器还通常采用较高时钟频率。综合这些机制，它非常适合控制密集、分支复杂、串行依赖明显或任务规模较小的程序部分。

4.3 图形处理器：面向高吞吐量的设计

图形处理器采用**面向吞吐量的设计** (*throughput-oriented design*)，主要优化大量相似任务的总完成速率。

4.3.1 大量较小的算术单元

图形处理器把更多芯片面积用于计算通道，因而能同时保持大量算术操作在执行流水线中。单个操作或单个线程的延迟可能高于中央处理器，但当工作规模足够大时，所有线程的总完成时间可以显著缩短。

4.3.2 深流水线与海量线程

图形处理器通过深流水线提高稳态吞吐量，并维护大量可执行线程。当一组线程等待长延迟事件，例如全局内存数据返回时，调度器可以选择另一组已经具备操作数的线程执行。

这个机制称为**延迟隐藏** (*latency hiding*)。它没有缩短那次内存访问本身的物理延迟；它通过让其他工作占用原本会空闲的执行周期，减少延迟对总体吞吐量的影响。若程序没有足够多的独立线程，或所有线程同时等待同一瓶颈，延迟就无法被有效隐藏。

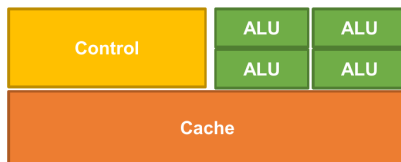
4.3.3 相对较小的缓存与较简单的控制

图形处理器也包含缓存，但相对于中央处理器，它把更大比例的面积用于算术单元、寄存器与线程执行资源。控制结构也偏向规则执行，因为最有效的工作负载通常让许多线程对不同数据执行相同或相近的指令序列。其时钟频率通常适中，性能主要来自并行宽度与总吞吐量，而不是极高频率。

Processor Design Approaches



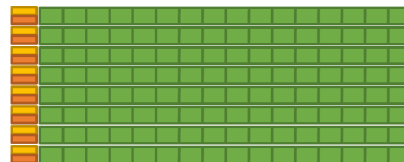
CPU: Latency-Oriented Design



- A few powerful **ALUs**
 - Reduced operation latency
- Large **caches**
 - Convert long latency memory accesses to short latency cache accesses
- Sophisticated **control**
 - Branch prediction to reduce control hazards
 - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency
- High clock frequency

ELECTRICAL & COMPUTER ENGINEERING

GPU: Throughput-Oriented Design



- Many small **ALUs**
 - Long latency, high throughput
 - Heavily pipelined for further throughput
- Small **caches**
 - More area dedicated to computation
- Simple **control**
 - More area dedicated to computation
- Massive number of threads to hide the very high latency!
- Moderate clock frequency

GRAINGER ENGINEERING

图 4 中央处理器的低延迟取向与图形处理器的高吞吐量取向

4.4 芯片面积分配反映了不同目标

图 4 的方块图不是精确的晶体管布局，而是面积预算的概念图。中央处理器把大量面积用于缓存和复杂控制，以减少少量线程的等待；图形处理器把更多面积复制为大量计算通道，以提高规则并行工作负载的总处理速率。

这种权衡可以从机会成本理解。若增加缓存和控制逻辑，就会减少可放置的算术通道；若增加算术通道，就必须接受较少的单线程优化资源。两类处理器都不是绝对“更快”，它们针对不同目标函数优化。

两张芯片照片可以进一步说明粒度差异：第十代英特尔酷睿处理器 (10th Gen Intel Core processor) 在 14 nm 工艺下包含 10 个中央处理器核心；英伟达 GK110 在 28 nm 工艺下标有 2880 个统一计算设备架构核心。较老工艺的 GK110 仍能容纳数量大得多的“核心”，正是因为这里的统一计算设备架构核心远小于完整中央处理器核心。该对比用于说明结构粒度，不能仅按数字推断两张芯片的通用性能。

表 7 中央处理器与图形处理器的高层对比

维度	中央处理器	图形处理器
主要目标 算术资源	降低单个线程或请求的延迟 少量、复杂、功能强的执行单元	提高大批相似任务的总吞吐量 大量、较小、深流水化的执行单元
缓存	较大，用于降低平均访存延迟	相对较小，把更多面积留给计算资源
控制 并发方式 典型优势部分	分支预测、数据转发等复杂机制 少量强线程，适度多线程 串行依赖、复杂控制、低延迟响应	偏向规则控制和大规模线程调度 海量线程，通过切换隐藏长延迟 数据并行、规则数值计算、高吞吐批处理

4.5 “核心”一词在两类处理器中并不等价

核心(*core*)在两类处理器中具有不同的结构粒度和语义。

中央处理器核心是相对完整的处理单元，通常包含多个执行单元、寄存器、至少一级私有缓存和复杂控制逻辑。它能够独立推进一个或多个通用指令流。

英伟达所称的**统一计算设备架构核心(CUDA core)**则是更细粒度的算术数据通路，可以近似理解为流式处理器中的一个算术执行单元。它不具备中央处理器核心那样完整的缓存与控制结构。因此，“10 个中央处理器核心”与“2880 个统一计算设备架构核心”不能按核心数量直接比较；两者的粒度、能力和组织方式完全不同。

4.6 高性能系统通常同时使用中央处理器与图形处理器

异构系统中的任务划分原则是：

- 对延迟敏感、难以并行化的串行部分由中央处理器执行；
- 对吞吐量敏感、含有大量相互独立工作的并行部分由图形处理器执行。

两类处理器在各自擅长的代码上可能相差十倍以上。该数值是概念性数量级说明，不是对任意程序的固定保证。实际加速取决于可并行比例、数据传输、内存访问、分支规则性、问题规模和实现质量。

一个典型异构应用的执行过程是：中央处理器读取与准备数据、决定计算阶段和发起图形处理器任务；图形处理器执行规模最大的并行计算；结果再被中央处理器用于后处理、控制或输出。两种设备之间的协同本身会引入数据移动与同步开销，因此正确的任务划分不仅

要比较计算速度，还要判断把任务迁移到加速器是否足以抵消这些额外成本。

关键结论

中央处理器与图形处理器的根本差异可以概括为：中央处理器用更多硬件资源缩短少数线程的关键路径，图形处理器用更多并行执行资源扩大单位时间内完成的工作总量。图形处理器编程的首要条件不是“有很多算术”，而是“有很多能够同时推进、且执行结构足够规则的算术”。

5 图形处理器的起源、统一计算设备架构与应用扩张

5.1 图形工作负载天然具有大规模数据并行性

图形处理器最初为图形渲染而设计，因此名称中的“图形”并不是历史偶然。屏幕包含大量像素，许多像素可以在相互独立或弱依赖的条件下执行相近的颜色、几何与纹理计算。把相同计算作用于大量不同数据元素，正符合高吞吐并行硬件的设计取向。

2007 年以前，图形处理器主要通过图形应用程序编程接口使用，例如**开放图形库 (OpenGL)** 与 **直接三维图形接口 (Direct3D)**。研究者若想让图形处理器完成非图形数值计算，必须把问题重新表述为图形流水线能够接受的形式，即把一般计算改写成“为某个像素着色的函数”，再把渲染输出解释为所需数值结果。

这种方法证明了图形处理器具有通用数值计算潜力，但编程过程高度间接，算法表达受图形接口约束。它促使硬件与软件接口共同演化。

GPU Origins



- GPUs were originally built for graphics
 - Hence the name, Graphics Processing Unit
 - Graphics workloads process a massive number of pixels in parallel
- Before 2007, the only way to program GPUs was using graphics APIs
 - e.g., OpenGL, Direct3D
 - Using GPUs for other than graphics meant reformulating a computation as a function that paints a pixel
- In 2007, NVIDIA released CUDA
 - A programming interface for using GPUs in a general-purpose way
 - Requires extensions to the GPU architecture



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 5 图形处理器从图形接口走向通用计算的历史转折

5.2 2007 年的统一计算设备架构转折

2007 年，英伟达发布**统一计算设备架构** (*Compute Unified Device Architecture, CUDA*)。它为图形处理器提供面向通用计算的编程接口，使程序员可以直接表达线程、内核函数、设备内存与数据传输，而不必把数值问题伪装成图形渲染。

这一转折通常称为**通用图形处理器计算** (*General-Purpose Computing on Graphics Processing Units, GPGPU*)。它包含两方面变化：

- 软件层面提供适合 C/C++ 程序员使用的语言扩展、运行时接口与工具链；
- 硬件层面增加或调整支持通用加载、存储、控制与数值计算的结构，使图形处理器不再只适合固定图形流水线。

统一计算设备架构 C/C++ 程序通常同时包含中央处理器代码和图形处理器代码：中央处理器端创建数据、分配设备资源并发起计算；图形处理器端执行由大量线程组成的**内核函数** (*kernel*)。这种主机端与设备端的协作构成统一计算设备架构程序的高层执行模型。

5.3 为什么沿用图形处理器，而不是从零设计另一种吞吐处理器

先进芯片的设计、验证、制造与软件生态建设成本极高，需要足够大的销量分摊固定成本。完全新建一种大规模并行处理器，即使技术上可行，也必须同时建立市场、编译器、工具、库和开发者生态。

图形处理器在并行计算进入主流之前已经拥有庞大的游戏与图形市场。这个既有装机基础带来三项先发优势：

1. 规模化销量能够分摊芯片研发与制造成本；
2. 图形工作负载已经推动硬件形成大规模并行执行能力；
3. 既有产品迭代、驱动与开发工具可以逐步扩展到科学计算和人工智能，而不必从零建立完整生态。

因此，图形处理器的成功来自技术结构与市场结构的共同作用。它既具备适合吞吐计算的硬件基础，也拥有其他专用加速器往往缺乏的持续商业规模。

5.4 数据中心与人工智能推动通用图形处理器计算扩张

英伟达 2019–2024 年按市场部门划分的收入曲线反映了通用图形处理器计算的扩张。图中数据中心收入在后期快速上升，这一变化与人工智能驱动的训练和推理需求密切相关。

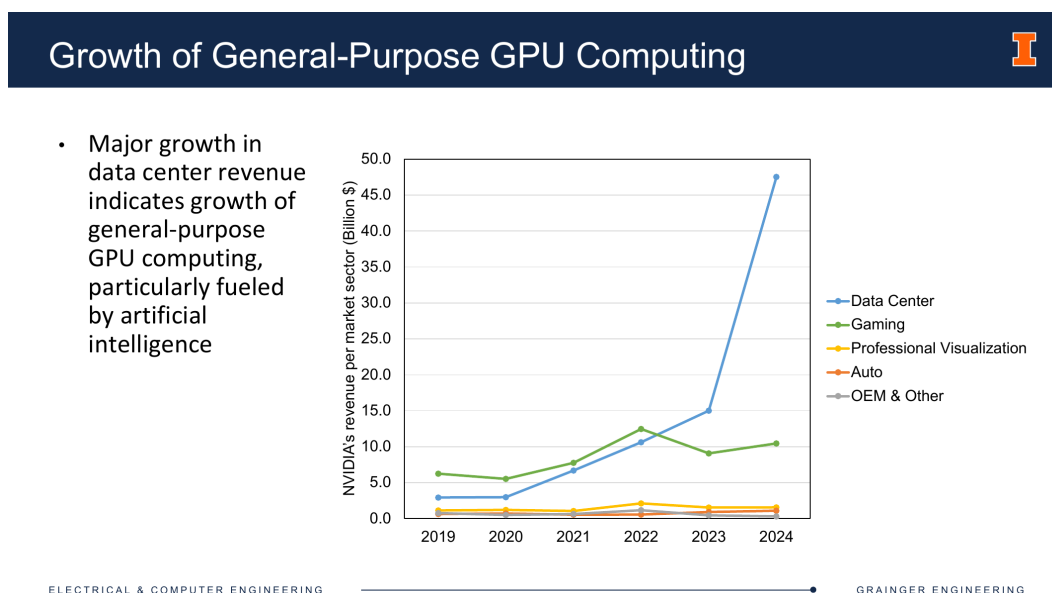


图 6 数据中心收入增长与通用图形处理器计算的扩张

该图反映了应用重心的变化：图形处理器仍服务于游戏与可视化，但数据中心、高性能计算和人工智能已成为其重要使用场景。模型训练需要执行大规模矩阵与张量运算；模型推理则把类似计算转化为持续服务需求。两者都可产生规模巨大且结构相对规则的并行工作。

5.5 人工智能计算需求的数量级增长

若干机器学习系统所需的浮点运算数量表明，从早期模型到大型语言与多模态模型，计算量跨越了许多数量级，最高约为 10^{24} 次浮点运算。

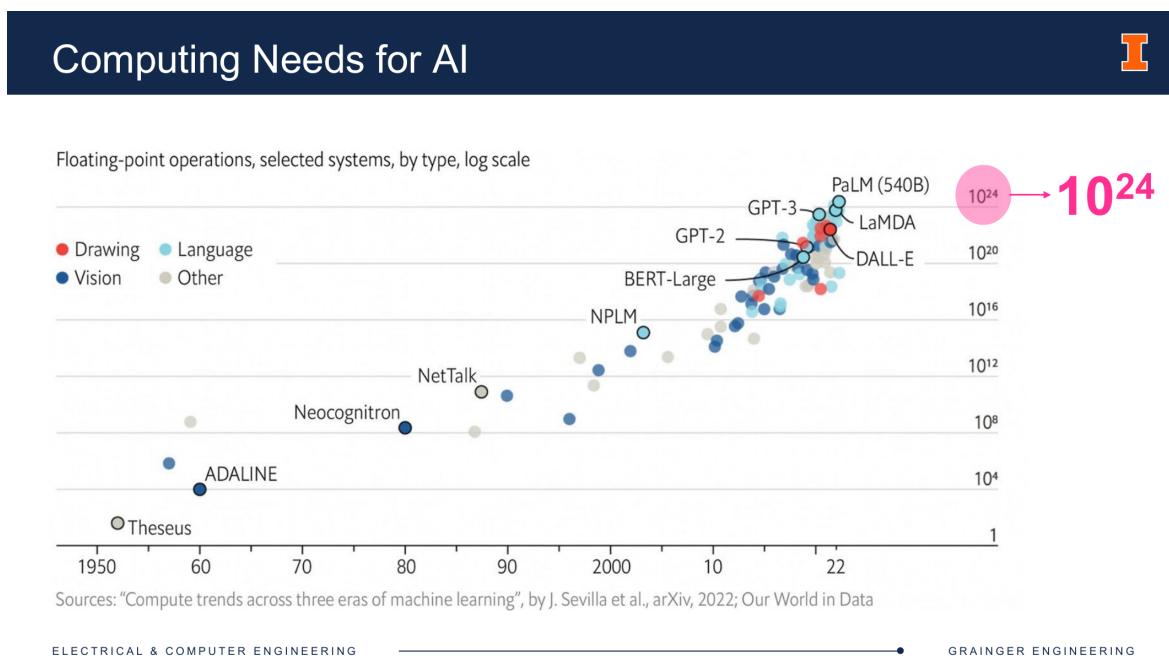


图7 人工智能系统计算需求增长趋势

对数坐标上的纵向小幅移动代表乘法级增长。例如从 10^{16} 增长到 10^{20} 不是增加四个单位，而是增加 10^4 倍。大模型计算需求的这种数量级变化，使单纯依靠单核性能提升不可行，也使高吞吐加速器、多设备并行与大规模系统调度成为必要条件。

5.6 超级计算机中的加速器普及

全球超级计算机五百强榜单依据大规模线性代数基准对系统进行排序，通常每半年更新一次，分别对应年中和年末的主要超级计算会议。2026 年 6 月榜单的前五名中，第一名使用自定义多核处理器；第二至第五名分别明确使用 AMD Instinct MI300A、AMD Instinct MI250X、英特尔数据中心图形处理器 Max 和英伟达 GH200 超级芯片。这个样本说明，顶级系统通常依靠大量加速器获得主要浮点计算能力。

Prevalence of GPUs in Supercomputers



• June 2026 Top-500 List

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	LineShine - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS, Shenzhen Cloud Computing Center Co., Ltd. National Supercomputing Centre in Shenzhen (NSCS), China	13,789,440	2,198.40	2,735.82	42,220
2	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL, United States	11,340,000	1,809.00	2,821.10	29,685
3	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory, United States	9,066,176	1,353.00	2,055.72	24,607
4	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory, United States	9,264,128	1,012.00	1,980.01	38,698
5	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, Bull EuroHPC/FZJ, Germany	4,801,344	1,000.00	1,226.28	15,794

<https://www.top500.org>

图 8 2026 年 6 月全球超级计算机五百强前五名

中央处理器仍负责操作系统、作业管理、控制流和部分串行计算；加速器负责高密度并行数值工作。要让几十年前为串行处理器编写的科学代码利用这些系统，必须重构算法、数据布局 and 并行分解。硬件峰值只是潜在能力，应用性能来自软件对硬件的有效映射。

5.7 并行编程框架之间的关系

统一计算设备架构是并行编程体系中的一种平台；其他常见框架分别提供跨设备编程、指令式卸载、分布式通信或共享内存并行等抽象。

表 8 常见并行编程框架

框架	主要目标	与统一计算设备架构的关系
统一计算设备架构	英伟达图形处理器的通用并行编程平台	本课程的主要编程环境。
开放计算语言 (OpenCL)	面向多类处理器与加速器的开放并行编程接口	提供另一类图形处理器并行编程框架。
开放加速器编程指令 (OpenACC)	通过编译指令把循环与数据区域卸载到加速器	可用于较高层次的加速器编程，本课程不以它为主线。
消息传递接口 (Message Passing Interface, MPI)	分布式内存系统中进程之间的显式通信	可与统一计算设备架构组合，在多个计算节点和多张图形处理器之间分配工作。
开放式多处理 (Open Multi-Processing, OpenMP)	共享内存系统中的线程级并行与编译指令	可与统一计算设备架构组合，协调单节点内的中央处理器线程及多设备工作。

单张图形处理器只解决一个设备内部的并行执行。若应用要扩展到多个节点，就需要额外机制完成进程间数据交换、任务划分和同步；消息传递接口承担这类分布式协调。若应用在单个共享内存节点内使用多个中央处理器线程或多个加速器，开放式多处理可以参与上层协调。统一计算设备架构、消息传递接口和开放式多处理因此可以处于同一应用的不同层次，而不是相互排斥的替代方案。

关键结论

图形处理器成为通用加速器的完整原因链为：图形渲染提供天然大规模数据并行性；游戏市场提供持续销量与研发投入；统一计算设备架构把硬件能力暴露为通用编程模型；科学计算与人工智能又提供了规模更大的规则并行工作负载。四个条件共同形成了今天的图形处理器计算生态。

6 并行化 workflows 与核心性能挑战

6.1 从“程序太慢”到可验证的并行实现

当已有程序运行过慢时，需要找出耗时部分，把适合的部分放到图形处理器上，并持续优化直到性能满足目标。并行化应形成测量驱动的闭环，而不能从“直接重写全部代码”开始。

6.1.1 建立正确的串行基线

首先保留一个能够产生正确结果的基线版本，并记录代表性输入上的运行时间。基线提供两种参照：一是验证并行版本是否保持相同计算语义，二是定义加速比的分母。没有可靠基线，后续的“更快”可能来自减少计算、降低精度或产生错误结果。

6.1.2 性能剖析并识别热点

使用性能剖析 (*profiling*) 工具测量各函数、循环或阶段的时间占比，找出真正主导总运行时间的热点 (*hotspot*)。如果某段代码只占总时间的 1%，即使把它加速 100 倍，对整体性能贡献也很有限；第 7 节将用阿姆达尔定律严格表达这一点。

剖析还应区分计算、内存访问、数据传输和同步等待。某一函数“耗时长”不一定表示算术量大，也可能表示它在等待数据或共享资源。

6.1.3 判断原算法能否暴露可扩展并行性

有些算法可以直接把独立迭代分配给不同线程；另一些算法包含长依赖链，原始形式不适合并行执行。此时需要采用或重新设计可扩展算法 (*scalable algorithm*)。可扩展性意味着：问题规模和处理资源增加时，可同时利用的有用工作也能增加，而串行协调、通信与资源争用不会迅速吞噬收益。

因此，并行化可能只是实现层面的映射，也可能要求算法层面的替换。若现有算法不适合并行执行，程序员需要选择结构完全不同的算法。

6.1.4 映射工作并优化数据布局

把工作划分为大量线程后，还要决定数据如何存放、何时移动以及哪些线程访问哪些数据，其目标是优化数据排列以最大化局部性。**局部性 (locality)** 表示程序在较短时间内重复访问相近或相关数据的倾向。良好的数据布局可以减少高延迟访问、提高缓存或片上存储的复用，并让大量线程更有效地使用全局内存带宽。

6.1.5 测量驱动的性能调优

性能调优 (performance tuning) 是迭代过程：修改实现、测量、解释瓶颈，再决定下一项修改。每次优化必须同时检查正确性和性能，不能仅依据代码看起来“更并行”就断言加速成立。

完整 workflow 可概括为：

正确基线 → 性能剖析 → 热点与并行性分析 → 算法或实现重构，
→ 数据布局优化 → 测量与验证 → 继续迭代。

最后还要检查代码的可移植性、可扩展性和可维护性。一个依赖特定设备偶然参数、只在单一输入上有效的实现，可能在下一代硬件或更大数据集上失效。

6.2 挑战一：大规模并行要求规则性

图形处理器最有效的执行形态是许多线程在同一阶段执行相同或相近的操作，只是处理不同数据。这种要求称为**规则性 (regularity)**。规则性包含至少三个层面：

- 工作结构规则：线程执行路径相近，不出现少数线程承担大量额外步骤；
- 数据访问规则：线程以可预测方式访问数据，避免大量随机、冲突或低复用访问；
- 阶段推进规则：线程组能在相近时间完成当前阶段，减少等待与闲置。

规则性不是说所有线程必须处理完全相同的数据，而是说硬件可以用共享的控制与调度机制高效驱动大量相似工作。若每个线程都执行截然不同的控制流，图形处理器复制大量算术通道的优势就难以兑现。

6.3 挑战二：负载均衡

设一个并行阶段包含 n 个任务，第 i 个任务的执行时间为 T_i 。如果下一阶段必须等所有任务完成，则该阶段的完成时间为

$$T_{\text{phase}} = \max_{1 \leq i \leq n} T_i. \quad (1)$$

式 (1) 表明，最快线程提前完成并不能缩短阶段时间；最慢线程决定何时能够继续。这就是**负载均衡 (load balance)** 的核心。

Parallel Computing Challenges



- Massive Parallelism demands Regularity
- Load Balance
 - The total amount of time to complete a parallel job is limited by the thread that takes the longest to finish



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 9 规则并行与负载不均衡

为了量化不均衡，可以定义一个解释性指标

$$E_{\text{load}} = \frac{\sum_{i=1}^n T_i}{n \max_i T_i}, \quad 0 < E_{\text{load}} \leq 1. \quad (2)$$

若所有任务耗时相同，则 $E_{\text{load}} = 1$ ；若少数任务远慢于其余任务，该比值会降低，表示大量并行资源在等待最慢任务时处于空闲状态。式 (2) 是用于理解负载不均衡的解释性指标；性能也可以用其他指标量化。

负载不均衡可能来自输入数据差异、分支路径差异、任务划分粒度过粗或某些任务争用更多资源。解决方向是重新分割任务、细化工作粒度或选择更规则的算法，使各执行单元承担的有效工作接近。

6.4 挑战三：全局内存带宽的理想值与实际值

全局内存带宽 (global memory bandwidth) 表示设备内存与处理器之间单位时间能够传输的数据量。硬件规格给出的峰值带宽是在理想访问模式下的上限，应用实际获得的**有效带宽 (effective bandwidth)** 可写为

$$B_{\text{eff}} = \frac{D_{\text{useful}}}{T}, \quad (3)$$

其中 D_{useful} 是完成目标计算所需的有效数据量， T 是相应时间。若程序重复搬运数据、访问模式分散或不能充分并行发起内存请求，则 B_{eff} 可能远低于峰值。

水坝与吸管的类比可以区分“理想”与“现实”：设备可能具备很宽的总带宽，但单个程序若组织不当，只能利用其中很小一部分。提高有效带宽依赖正确的内存模型、数据复用和访问模式。

6.5 挑战四：冲突访问导致串行化与延迟

当大量线程同时访问某个受限或关键资源时，请求必须排队，原本并行的工作被迫部分**串行化 (serialization)**。国际机场入境检查可以类比这种瓶颈：许多航班可以同时到达，但若所有旅客必须通过数量有限的检查窗口，检查阶段就成为窄口，前面的并行到达能力无法转化为端到端吞吐量。

Parallel Computing Challenges (cont'd)

- Global Memory Bandwidth
 - Ideal vs. Reality

- Conflicting Data Accesses Cause Serialization and Delays
 - Massively parallel execution cannot afford serialization
 - Contentions in accessing critical data causes serialization

Ideal



Reality





ELECTRICAL & COMPUTER ENGINEERING
GRAINGER ENGINEERING

图 10 全局内存带宽与关键资源争用

从系统角度看，争用会产生两类损失：

1. 请求排队，单个访问的等待时间增加；
2. 等待资源的线程不能推进，执行通道可能空闲，整体吞吐量下降。

海量并行系统对串行化尤其敏感。若只有一个线程，某个小的串行步骤只是该线程的一部分；若有成千上万线程同时汇聚到同一关键资源，该步骤会把大量并行执行压缩为一条队列。同步、共享数据更新与内存访问设计都必须控制这种汇聚效应。

6.6 四类挑战之间的耦合

规则性、负载均衡、带宽和争用不是相互独立的问题。一个不规则算法可能同时造成：不同线程工作量差异、数据访问失去局部性、多个线程争用热点数据，以及难以隐藏的等待。相反，规则的数据并行结构通常更容易均匀划分工作，也更容易形成高效访存。

关键结论

并行化的目标不是把串行语句机械地复制到更多线程，而是构造一种硬件能够持续推进的工作形态：线程数量足够多，执行路径足够规则，任务负载相近，数据访问能够利用带宽，并且不存在把全部线程重新压回串行队列的关键资源。

7 阿姆达尔定律：整体加速比的串行上界

7.1 问题设定与符号

考虑一个固定规模的应用，其原始串行执行时间为 t 。把原始时间按能否被目标并行化方法加速分成两部分：

- 可并行部分占比为 p ，原始耗时为 pt ；
- 不可并行部分占比为 $1 - p$ ，原始耗时为 $(1 - p)t$ ；
- 可并行部分经过并行化后获得局部加速比 s ，因此其新耗时为 pt/s 。

这里 $0 \leq p \leq 1$ ， $s \geq 1$ 。该模型假设问题规模固定，并暂时不单独计入内核启动、数据传输、同步与通信等额外开销。

7.2 总执行时间的推导

不可并行部分没有被加速，因此仍需 $(1 - p)t$ ；可并行部分缩短为 pt/s 。新程序的总执行时间为

$$T_{\text{new}} = (1 - p)t + \frac{pt}{s} = \left(1 - p + \frac{p}{s}\right)t. \quad (4)$$

整体加速比定义为原始执行时间与新执行时间之比：

$$S(p, s) = \frac{t}{T_{\text{new}}} = \frac{t}{\left(1 - p + \frac{p}{s}\right)t} = \boxed{\frac{1}{1 - p + \frac{p}{s}}}. \quad (5)$$

原始时间 t 在分子与分母中约去，因此在这一固定比例模型中，整体加速比只由可并行比例 p 和局部加速比 s 决定。

7.3 并行部分无限快时的上界

即使可并行部分被理想化为瞬间完成，即 $s \rightarrow \infty$ ，仍有

$$\lim_{s \rightarrow \infty} S(p, s) = \frac{1}{1 - p}. \quad (6)$$

式 (6) 是阿姆达尔定律最重要的含义：整体加速比由未被加速的串行比例限制。可并行部分继续变快时，它在总时间中的占比越来越小，最终总时间几乎全部由串行部分构成。

Amdahl's Law



- In general, if an application has:
 - t sequential execution time
 - p fraction of execution that is parallelizable
 - s speedup achieved on the parallelizable part
- What is the overall speedup of the application?

$$t_{parallel} = (1 - p) * t + \frac{p * t}{s} = \left(1 - p + \frac{p}{s}\right) * t$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{t}{\left(1 - p + \frac{p}{s}\right) * t} = \frac{1}{1 - p + \frac{p}{s}} \xrightarrow{s \rightarrow \infty} \frac{1}{1 - p}$$

ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 11 阿姆达尔定律推导

7.4 示例：90% 可并行，局部加速 1000 倍

例题

原程序串行运行时间为 $t = 100\text{ s}$ ，可并行比例为 $p = 0.9$ ，可并行部分在图形处理器上获得 $s = 1000$ 的加速。求新执行时间与整体加速比。

串行部分原本占 10%，因此保持为

$$(1 - p)t = (1 - 0.9) \times 100\text{ s} = 10\text{ s}.$$

可并行部分原本耗时 90 s，加速 1000 倍后为

$$\frac{pt}{s} = \frac{0.9 \times 100\text{ s}}{1000} = 0.09\text{ s}.$$

所以

$$T_{\text{new}} = 10\text{ s} + 0.09\text{ s} = 10.09\text{ s},$$

$$S = \frac{100\text{ s}}{10.09\text{ s}} \approx 9.91.$$

尽管局部加速比达到 1000，整体加速比仍只有约 9.91。若把可并行部分变为无限快，整体

上界也只是

$$S_{\max} = \frac{1}{1 - 0.9} = 10.$$

7.5 串行比例对上界的高度敏感性

表 9 不同可并行比例对应的理想整体加速上界

可并行比例 p	串行比例 $1 - p$	$s \rightarrow \infty$ 时的上界 $1/(1 - p)$
0.50	0.50	2
0.90	0.10	10
0.95	0.05	20
0.99	0.01	100
0.999	0.001	1000

要获得数量级更高的整体加速，串行比例必须按同样数量级下降。理想情况下，若希望整体加速达到 S_{target} ，至少需要

$$1 - p \leq \frac{1}{S_{\text{target}}}.$$

例如，100 倍加速要求串行比例不超过 1%；有限的局部加速和额外开销会使实际要求更严格。许多大数据集应用的可并行比例可以超过 99%，因此超过 100 倍的加速在合适条件下可以实现。

7.6 为什么先做性能剖析

假设某函数占总时间的比例为 p ，对该函数的任何局部优化都受式 (5) 约束。因此，性能剖析的作用不是只找“看起来复杂”的代码，而是确定哪些部分具有足够大的时间占比。优化低占比部分，即使局部加速很高，整体收益也会被阿姆达尔定律压缩。

反过来，当并行部分已经极快时，原先不起眼的串行初始化、数据准备、输出或同步会成为新的热点。性能优化因此具有阶段性：每消除一个瓶颈，时间分布都会改变，需要重新剖析，而不能永久沿用最初的热点排序。

7.7 模型边界与现实开销

实际异构程序通常还包含图形处理器内核启动、中央处理器与图形处理器之间的数据移动、同步、资源争用和调度开销。若把这些额外时间记为 T_{overhead} ，则更现实的表达为

$$S_{\text{real}} = \frac{t}{(1-p)t + \frac{pt}{s} + T_{\text{overhead}}}. \quad (7)$$

式 (7) 在基本模型中加入了显式开销。它说明，即使 p 很大，问题规模过小也可能无法抵消固定开销；提高局部内核速度之后，数据移动与同步可能主导总时间。这与第 3 式所描述的有效带宽问题以及资源争用问题相互关联。

关键结论

阿姆达尔定律给出的优化顺序是：先找到占比最大的可加速部分，同时持续削减不可并行与协调开销；局部加速达到很高后，必须重新测量，因为瓶颈会迁移到之前占比很小的部分。整体性能永远由完整执行路径决定。

8 综合框架与关键结论

8.1 硬件演化到软件责任的完整链条

硬件演化与软件责任构成以下统一框架：

1. 摩尔定律长期提高晶体管密度，登纳德缩放长期提高时钟频率；两者共同形成早期单核性能快速增长。
2. 约在 2005–2006 年，漏电流、功耗与散热约束使频率增长趋于停滞，但晶体管数量仍能继续增加。
3. 新增晶体管被组织为多核、向量执行资源和专用加速器。现代系统因此具有显著并行性与异构性。
4. 中央处理器面向低延迟，适合串行依赖和复杂控制；图形处理器面向高吞吐量，适合大量规则且相互独立的计算。
5. 图形处理器原有的游戏市场和图形并行结构提供了经济与技术基础；统一计算设备架构进一步把它转化为通用计算平台。

6. 应用必须显式暴露并行性。性能依赖算法可扩展性、线程规则性、负载均衡、数据布局、有效带宽和争用控制。
7. 任意局部加速最终都受完整应用中未加速部分限制，阿姆达尔定律给出这一上界。

8.2 选择中央处理器或图形处理器时的判断顺序

面对一个待加速代码段，可以按下列问题逐步判断：

1. 它在完整程序中占多少时间？占比过低时，局部优化的整体收益受限。
2. 是否存在大量相互独立或弱依赖的任务？任务数量必须足以填充图形处理器。
3. 各任务是否执行相同或相近的操作？高度不规则控制流会降低并行执行效率。
4. 每个任务的工作量是否接近？最慢任务会决定并行阶段完成时间。
5. 数据访问是否规则、具有局部性，并能利用全局内存带宽？
6. 把数据移动到图形处理器并启动内核的开销，是否小于并行计算所节省的时间？
7. 实现能否在更大输入和后续硬件上保持正确性、性能与可维护性？

只有当这些问题的总体答案支持加速器映射时，图形处理器才可能产生端到端收益。对于串行依赖明显、任务规模很小或数据移动占主导的代码，中央处理器可能更合适。

8.3 高频概念辨析

容易形成的误解	准确理解
摩尔定律等同于处理器性能翻倍。	历史性能增长来自晶体管密度、频率缩放和处理器结构改进的共同作用；摩尔定律主要描述晶体管密度趋势。
处理器核心数量增加会自动加速串行程序。	串行指令流通常只使用一个核心；程序必须暴露并行任务并承担调度、通信和同步成本。
图形处理器的单个线程一定比中央处理器线程快。	图形处理器优化大批线程的总吞吐量，单线程延迟可能更高。
统一计算设备架构核心与中央处理器核心可以按数量直接比较。	两者粒度不同；统一计算设备架构核心更接近细粒度算术执行通道。
硬件峰值内存带宽就是程序能够获得的带宽。	实际有效带宽取决于数据布局、访问模式、复用、并发程度和争用。

容易形成的误解	准确理解
并行部分加速 1000 倍会让整个程序加速 1000 倍。	整体加速由 $S = 1/(1 - p + p/s)$ 决定，未加速部分最终成为上界。
问答系统使用课程材料，因此答案必然正确。	检索增强生成只能改善相关性；检索遗漏、隐含假设与生成错误仍可能导致错误答案。

8.4 “高性能”的操作性定义

高性能需要放在完整软件工程目标中理解。并行实现应满足：

- **功能正确**：与参考语义一致，能够通过规定的数据集与问题检查；
- **端到端加速**：完整应用时间缩短，而不是只报告某个局部内核的理想时间；
- **可解释**：能够用并行度、访存、负载、串行比例等因素解释性能变化；
- **可扩展**：问题规模或硬件资源增加时仍能利用更多并行能力；
- **可维护与可移植**：实现结构清晰，不依赖难以复现的偶然参数，并能适应硬件代际变化。

这五项要求也解释了项目评分为何同时关注功能、编码风格和性能。单独追求某一项会产生失衡：正确但极慢的实现没有利用加速器价值；快速但错误的实现改变了问题；高度手工特化但不可维护的实现难以跨代延续。

8.5 阅读与操作任务

需要完成以下事项：

- 阅读教材第 1 章；
- 阅读《统一计算设备架构编程指南》第 1.1 节“导论”；
- 尽快创建高等科学与工程研究计算环境账户并取得 Delta 访问权限；
- 配置个人代码仓库，完成课程发布代码的同步流程；
- 在账户可用后验证安全外壳协议登录、编译节点与计算节点作业提交流程。

关键结论

软件可持续性要求算法与库具备可扩展性和可移植性，从而跨越多代硬件；其价值来自对并行结构和性能瓶颈的清晰建模，而不是依赖某一代设备的偶然峰值。